LayerZero Stellar

Security Assessment

July 14th, 2026 — Prepared by OtterSec

Andreas Mantzoutas andreas@osec.io

Renato Eugenio Maria Marziano renato@osec.io

Table of Contents

01 — Executive Summary

Overview

LayerZero engaged OtterSec to assess the `stellar` program. This assessment was conducted between February 5th and March 20th, 2026. A follow-up audit was conducted on July 14th, 2026. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 12 findings throughout this audit engagement.

In particular, we identified a vulnerability where the alert API allows negative gas and value inputs even though those fields should never be negative (OS-LZS-ADV-00). Additionally, in LzExecutor, the logic for setting a helper is currently admin-controlled even though it configures a trusted authorization intermediary, giving admins more power than is likely intended (OS-LZS-ADV-01). Furthermore, optional fee configuration is currently allowed in storage, but execution paths later treat those values as mandatory, so fee-paying sends can revert when the required recipient or `ZRO` fee library is unset (OS-LZS-ADV-02).

We also made suggestions to utilize checked arithmetic or saturating operations where applicable to mitigate against overflows (OS-LZS-SUG-02), and recommended allowing the endpoint contract to perform the transfer itself, rather than expecting the caller to have already sent the assets (OS-LZS-SUG-03). Moreover, the same signer set is reutilized across chains, therefore, signed payloads must be unambiguously scoped to their intended chain and signing scheme to avoid potential collisions (OS-LZS-SUG-05). We further advised improving the documentation for the role macros, clearly contrasting their guarantees and functionalities to avoid any ambiguities (OS-LZS-SUG-07).

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/LayerZero-Labs/monorepo-external. This audit was performed against commit c1fb16e. A follow-up audit was performed against d20f6fa.

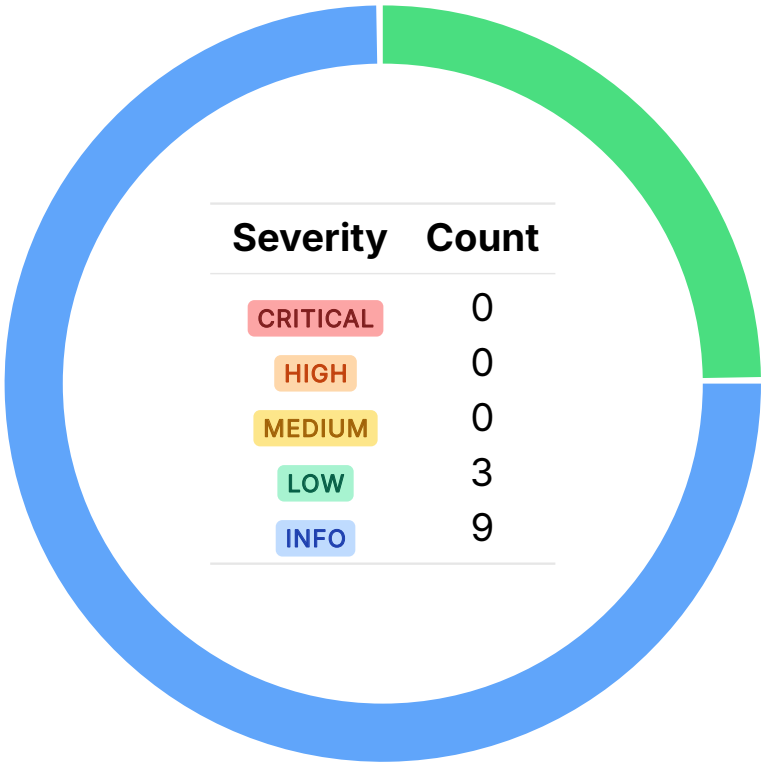
A brief description of the program is as follows:

Name	Description
stellar	Hardens the Stellar EndpointV2 / ULN-302 flow by enforcing contract-only senders, validating quoted fees, and reordering fee transfers after state updates to reduce risk. It also includes internal refactors.

03 — Findings

Overall, we reported 12 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	0
LOW	3
INFO	9

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-LZS-ADV-00	LOW	RESOLVED	The alert APIs allow negative `gas` and `value` inputs even though those fields should never be negative.
OS-LZS-ADV-01	LOW	RESOLVED	`set_executor_helper` is currently admin-controlled even though it configures a trusted authorization intermediary, giving admins more power than is likely intended.
OS-LZS-ADV-02	LOW	RESOLVED	In the `oft_fee` extension, `fee_deposit_address` is optional in storage, but nonzero fee collection requires it to exist, so the contract may enter an invalid fee-enabled state that results in a revert. Similarly, the treasury will panic if users attempt `ZRO` payment while the `ZRO` fee library is unset.

Possibility of Negative Alert Execution Parameters LOW OS-LZS-ADV-00

Description

The alert APIs (`lz_receive_alert` and `lz_compose_alert`) accept `gas` and `value` as signed `i128`, even though both fields are inherently non-negative execution parameters. This allows impossible negative values to be emitted in `lz_receive_alert` and `lz_compose_alert`, weakening the integrity of on-chain monitoring and debugging data.

Remediation

Enforce non-negativity for `gas` and `value`.

Patch

Resolved in #2915.

Excessive Admin Privileges LOW OS-LZS-ADV-01

Description

`set_executor_helper` in `LzExecutor` currently requires only admin authorization, but the helper configuration influences executor-side authorization flow rather than only operational parameters. Because the helper is later trusted by auth-context validation, the ability to set it is effectively the ability to define a privileged execution intermediary. Granting this power to admins may be excessive, since an admin may both register a helper they control and immediately utilize its whitelisted functions to exercise privileged behavior. Because helper configuration changes are infrequent but high impact, they are better suited to owner-only governance rather than a broader admin role.

>_ *protocol/stellar/contracts/workers/executor/src/executor.rs* RUST

```rust
pub fn set_executor_helper(env: &Env, admin: &Address, helper: &Address, allowed_functions:
    ↪  &Vec<Symbol>) {
    require_admin_auth::<Self>(env, admin);
    ExecutorStorage::set_executor_helper(
        env,
        &crate::storage::ExecutorHelperConfig {
            address: helper.clone(),
            allowed_functions: allowed_functions.clone(),
        },
    );
}
```

Remediation

Restrict `set_executor_helper` to `only_auth` to reduce centralization risk and enforce clearer privilege separation.

Patch

Resolved in #3284.

Potential Reverts Due to Optional Fee Configuration Logic LOW OS-LZS-ADV-02

Description

The `oft_fee` extension allows `fee_deposit_address` to be unset in storage, as `__fee_deposit_address` returns `Option<Address>`, so `None` is treated as a valid stored state. But in `__charge_fee`, if `fee_amount != 0`, the code immediately calls `unwrap_or_panic`. That implies any route with a nonzero effective fee will revert if `fee_deposit_address` is unset in storage. This creates a configuration-state mismatch where fee collection may be enabled while the required recipient address is missing.

>_ *protocol/stellar/contracts/oapps/oft/src/extensions/oft_fee.rs* RUST

```rust
fn __charge_fee(env: &Env, token: &Address, from: &Address, fee_amount: i128) {
    if fee_amount != 0 {
        let fee_deposit =
            Self::__fee_deposit_address(env).unwrap_or_panic(env,
                ↪  OFTFeeError::InvalidFeeDepositAddress);
        TokenClient::new(env, token).transfer(from, &fee_deposit, &fee_amount);
    }
}

fn __fee_deposit_address(env: &Env) -> Option<Address> {
    OFTFeeStorage::fee_deposit_address(env)
}
```

This issue also exists in `treasury`, when `pay_in_zro` is true. In this case, Stellar treasury fee quoting calls `expect_zro_fee_lib_client`, which panics if `zro_fee_lib` is unset, resulting in `quote_treasury` to abort. This is stricter than the EVM behavior, where the `ULN` treasury path effectively returns zero treasury fee instead of reverting.

>_ *protocol/stellar/contracts/message-libs/treasury/src/treasury.rs* RUST

```rust
/// Returns the ZRO fee library client, panics if not set.
fn expect_zro_fee_lib_client(env: &Env) -> ZroFeeLibClient<'static> {
    let fee_lib = Self::zro_fee_lib(env).unwrap_or_panic(env, TreasuryError::ZroFeeLibNotSet);
    ZroFeeLibClient::new(env, &fee_lib)
}
```

Remediation

The invariant should be enforced explicitly by preventing nonzero fees without a configured `fee_deposit_address`, or by configuring a default fee receiver. Also, avoid panicking when `zro_fee_lib`

is unset. Instead, treat the `ZRO` treasury fee as zero. This aligns with the `EVM` implementation and avoids unnecessary hard failures.

Patch

LayerZero acknowledged this issue and noted that it does not pose an issue, as the send operation fails with an explicit error message.

05 — General Findings

Here, we present a discussion of general findings identified during our audit. While these findings do not pose an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-LZS-SUG-00	`BlockedMessageLib` gets TTL-extension code from the macro, but without a `__constructor` its default TTL configuration is never initialized, so the extension logic never takes effect.
OS-LZS-SUG-01	Stellar `native_drop_and_execute` has different failure semantics from EVM `nativeDropAndExecute302`, wherein, if `lz_receive` fails, the native drop is rolled back instead of persisting.
OS-LZS-SUG-02	There are several instances of unchecked arithmetic which may result in an overflow that should be addressed.
OS-LZS-SUG-03	`pay_messaging_fees` utilizes the endpoint contract's entire token balance as the fee supplied for the current send, so leftover or mistakenly sent funds may be unintentionally paid out or refunded as part of another user's transaction.
OS-LZS-SUG-04	The `oapp` macro implicitly ties RBAC generation to the `core` option, even though other default-generated traits such as `options_type3` may still depend on RBAC.
OS-LZS-SUG-05	The same signer set is reutilized across chains for a given DVN, but the signed preimages differ between chains, potentially leading to cross-chain message forgeries.
OS-LZS-SUG-06	Stellar alerts are not as tightly bound to actual failed `lz_receive` attempts as in EVM, so an emitted alert is not a guarantee of a real execution failure.

OS-LZS-SUG-07	`has_role` only appears to check whether an address has a role, whereas `only_role` also enforces `require_auth`, so they provide materially different guarantees, which is not obvious in their documentation currently.
OS-LZS-SUG-08	The `oapp-macros` documentation incorrectly shows `#[ownable]` decorating `impl OAppCore` blocks, which is the incorrect macro target, and may mislead users.

Absence of TTL Initialization Path in BlockedMessageLib OS-LZS-SUG-00

Description

`contract_impl` injects TTL extension logic into callable methods, but that logic only works if default TTL configuration was previously initialized in a `__constructor`. Since `BlockedMessageLib` has no `__constructor`, `init_default_ttl_configs(env)` is never injected or run, so the inserted extension path becomes a no-op. As a result, the contract may appear TTL-managed while its instance TTL is never actually refreshed during utilization.

>_ *protocol/stellar/contracts/common-macros/src/contract_ttl.rs* RUST

```rust
pub fn contractimpl_with_ttl(attr: TokenStream, input: TokenStream) -> TokenStream {
    let mut impl_block: ItemImpl = syn::parse2(input).unwrap_or_else(|e| panic!("failed to parse
        ↪ impl block: {}", e));

    let is_trait_impl = impl_block.trait_.is_some();

    for item in &mut impl_block.items {
        [...]
        if method.sig.ident == "__constructor" {
            // Inject default TTL config initialization in constructor
            method.block.stmts.insert(0, init_default_ttl_configs_stmt(&env_param));
        } else {
            // Inject TTL extension at the start of other methods
            method.block.stmts.insert(0, extend_instance_ttl_stmt(&env_param));
        }
    }
    [...]
}
```

Remediation

Ensure that any contract utilizing the TTL macro always initializes default TTL configuration.

Stellar Native Drop Rollback Semantics OS-LZS-SUG-01

Description

In the EVM implementation, `nativeDropAndExecute302` preserves the native drop even when `lzReceive` fails, because the receive path is wrapped in `try` / `catch` after the drop occurs. By contrast, Stellar `native_drop_and_execute` is effectively atomic, so a failed `lz_receive` reverts the whole call and the native drop does not persist. If backend services compensate by reissuing the native drop separately, it should be documented explicitly.

Remediation

Explicitly document that a failed `lz_receive` rolls back the native drop, and that native-drop persistence after execution failure is achieved only through separate backend implementations.

Code Overflow OS-LZS-SUG-02

Description

`PriceFeed::estimate_fee_with_default_model` and `ExecutorFeeLib::convert_and_apply_premium_to_value` multiply `u128` operands before division, so large `remote_fee * price_ratio` or `value * ratio` intermediates may overflow. With realistic cross-chain ratios these exceed `u128::MAX` resulting in an overflow, and because Rust overflow checks are enabled, these overflows will create a panic, resulting in a denial-of-service scenario as the panic will result in `quote` and any dependent `send` paths to revert on affected routes. The issue is overlooked in tests because they utilize one-to-one ratios that keep intermediate products small and do not reflect real-world pricing scenarios. A lower `priceRatioDenominator` may reduce overflow likelihood by shrinking ratio numerators.

>_ protocol/stellar/contracts/workers/price-feed/src/price_feed.rs RUST

```rust
fn estimate_fee_with_default_model(env: &Env, dst_eid: u32, calldata_size: u32, gas: u128) ->
    ↪ (u128, u128) {
    [...]
    let remote_fee = (gas_for_calldata + gas) * (price.gas_price_in_unit as u128);
    let fee = (remote_fee * price.price_ratio) / Self::get_price_ratio_denominator(env);
    (fee, price.price_ratio)
}
```

>_ protocol/stellar/contracts/workers/executor-fee-lib/src/executor_fee_lib.rs RUST

```rust
fn convert_and_apply_premium_to_value(env: &Env, value: u128, ratio: u128, denom: u128,
    ↪ multiplier_bps: u32) -> i128 {
    if value == 0 {
        return 0;
    }
    let converted = (value * ratio) / denom;
    safe_u128_to_i128(env, (converted * multiplier_bps as u128) / BPS_DENOMINATOR)
}
```

Similarly in `RateLimiter::calculate_decayed_in_flight`, `decay` computation multiplies two potentially large values before division. It performs multiplication before division, so the intermediate `elapsed * limit` product may overflow `u128` bounds before division. However, with 7-decimal Stellar tokens, this condition is unlikely to be reached in practice.

>_ *protocol/stellar/contracts/oapps/oft/src/extensions/rate_limiter.rs* RUST

```rust
fn calculate_decayed_in_flight(env: &Env, state: &RateLimitState) -> i128 {
    [...]
    let elapsed = timestamp - state.last_update;
    let decay = (elapsed as i128) * state.config.limit / (state.config.window_seconds as i128);
    [...]
}
```

Also, `BufferReader::skip` computes `self.pos + len` without checked arithmetic, so the cursor calculation itself may overflow before `seek` validates the result, giving rise to an unexpected panic.

>_ *protocol/stellar/contracts/utils/src/buffer_reader.rs* RUST

```rust
/// Advances the position by `len` bytes without reading.
pub fn skip(&mut self, len: u32) -> &mut Self {
    self.seek(self.pos + len)
}
```

Remediation

Ensure to utilize checked arithmetic or saturating operations where applicable to mitigate against overflows.

Endpoint Utilizes Ambient Fee Balance OS-LZS-SUG-03

Description

`endpoint_v2::pay_messaging_fees` treats the endpoint's full token balance as the fee supplied for the current send, rather than tracking the amount actually provided by that transaction. That is risky because the endpoint is not guaranteed to hold only the exact fee amount for the in-flight send. The existence of `recover_token` strongly suggests the contract may at times receive mistaken transfers or retain stray balances. As a result, any leftover or mistakenly sent funds already sitting in the endpoint may be unintentionally utilized to pay message fees or refunded to the current caller's `refund_address`.

>_ *protocol/stellar/contracts/endpoint-v2/src/endpoint_v2.rs* RUST

```rust
fn pay_messaging_fees(
     env: &Env,
     pay_in_zro: bool,
     native_fee_recipients: &Vec<FeeRecipient>,
     zro_fee_recipients: &Vec<FeeRecipient>,
     refund_address: &Address,
 ) -> MessagingFee {
    let this_contract = env.current_contract_address();
    let mut fee_paid = MessagingFee { native_fee: 0, zro_fee: 0 };
    // Pay native fees
    let native_token_client = TokenClient::new(env, &Self::native_token(env));
    let mut native_fee_supplied = native_token_client.balance(&this_contract);
    [...]
}
```

Remediation

Ensure the endpoint pulls or transfers the exact fee amount for the current send instead of inferring it from `balance(this_contract)`.

Patch

The LayerZero team acknowledged this issue, stating that the behavior is intentional and aligned with the EVM implementation.

Implicit RBAC Generation Coupling OS-LZS-SUG-04

Description

The `oapp` macro currently treats RBAC generation as an implicit side effect of the `core` option, even though other generated traits such as `options_type3` also depend on `RoleBasedAccessControl`. A user may reasonably assume the core flag controls only `OAppCore`, when in practice it also determines whether RBAC is auto-implemented. Other generated traits, such as `OAppOptionsType3`, also extend `RoleBasedAccessControl`. So if a user selects a custom `core` implementation but keeps the default `options_type3` implementation, they may unexpectedly lose the auto-generated RBAC implementation that `options_type3` still depends on.

>_ *protocol/stellar/contracts/oapps/oapp-macros/src/generators.rs* RUST

```rust
fn generate_oapp_core(name: &Ident) -> TokenStream {
    quote! {
        use oapp::oapp_core::OAppCore as _;
        use utils::rbac::RoleBasedAccessControl as _;

        #[soroban_sdk::contractimpl(contracttrait)]
        impl oapp::oapp_core::OAppCore for #name {}

        #[soroban_sdk::contractimpl(contracttrait)]
        impl utils::rbac::RoleBasedAccessControl for #name {}
    }
}
```

Remediation

Explicitly document that auto-generating `core` currently also auto-generates `RoleBasedAccessControl`, and that opting out of generated core may require the user to implement RBAC themselves when utilizing other default traits that depend on it. Additionally, an improved design may be to add a separate RBAC flag to enable or disable auto-implementing RBAC. Alternatively, RBAC generation may be made conditional on whether any selected generated trait requires it. For example, if `core` is custom but `options_type3` is default, auto-generating the RBAC implementation will keep the default-generated trait set coherent.

Patch

The LayerZero team acknowledged this issue, noting that it is a rare scenario. They stated that developers may manually implement both traits if required and will update the documentation to clarify this behavior.

Differing Cross-Chain Signed Preimage Formats for DVN Multisigs OS-LZS-SUG-05

Description

There is a potential issue concerning cross-chain signature domain separation. Because the same signer set is reutilized across multiple chains, the safety of the overall design also depends on whether a payload signed for one chain could ever be interpreted as valid authorization on another chain. Ethereum signs an `ERC-191` -prefixed hash-of-a-hash, using this payload:

>_ *layerzero-v2/evm/messagelib/contracts/uln/dvn/DVN.sol* SOLIDITY

```solidity
function hashCallData(
    uint32 _vid,
    address _target,
    bytes calldata _callData,
    uint256 _expiration
) public pure returns (bytes32) {
    return keccak256(abi.encodePacked(_vid, _target, _expiration, _callData));
}
```

While the Stellar implementation utilizes a formally different preimage:

>_ *protocol/stellar/contracts/workers/dvn/src/dvn.rs* RUST

```rust
fn hash_call_data(env: &Env, vid: u32, expiration: u64, calls: &Vec<Call>) -> BytesN<32> {
    let mut writer = BufferWriter::new(env);
    let data =
        ↪ writer.write_u32(vid).write_u64(expiration).write_bytes(&calls.to_xdr(env)).to_bytes();
    env.crypto().keccak256(&data).into()
}
```

Remediation

Ensure that signed payloads remain consistently formatted and signed across chains.

Alerts Not Strictly Failure-Coupled OS-LZS-SUG-06

Description

In EVM, `execute302` itself performs `lzReceive` and immediately emits `lzReceiveAlert` only if that exact call reverts, so an alert implies a real failed execution attempt. In Stellar, alert emission is not as tightly coupled to the receive attempt, so the same guarantee does not hold. Thus, emitting an alert does not necessarily imply that the corresponding receive call failed. This may mislead off-chain systems that treat alerts as authoritative failure signals for retries, monitoring, or incident handling.

Remediation

Enforce execution through the `executor_helper` and incorporate logic within the helper to invoke `try_lz_receive`; in case of failure, it should call `lz_receive_alert`.

Patch

The LayerZero team acknowledged this issue, and they opted for a two-step alert submission process because its off-chain systems already support that workflow, rendering the integration straightforward.

Code Clarity OS-LZS-SUG-07

Description

`has_role` macro appears to check only role membership, while `only_role` also enforces caller authorization via `require_auth`, so the two are not interchangeable. Without an explicit comment stating that `has_role` does not perform auth, developers may mistakenly treat it as a full access-control guard.

Remediation

Explicitly state in the documentation for `has_role` that it does not perform `require_auth`, clearly contrasting it with `only_role` to prevent them from being utilized interchangeably.

Improper Documentation OS-LZS-SUG-08

Description

The `oapp-macros` documentation currently places `#[common_macros::ownable]` on `impl OAppCore` blocks, which appears to be an invalid macro target and gives users the wrong integration pattern. This should be corrected across the docs so ownership is shown on the proper item type and users do not infer that contract-level auth macros decorate trait `impl` blocks.

>_ *protocol/stellar/contracts/oapps/oapp-macros/src/lib.rs* RUST

```rust
//! #[contractimpl(contracttrait)]
//! #[common_macros::ownable]
//! impl OAppCore for MyOApp {
//!     fn oapp_version(_env: &Env) -> (u64, u64) {
//!         (1, 1) // Custom version
//!     }
//! }

//! #[contractimpl(contracttrait)]
//! #[common_macros::ownable]
//! impl OAppCore for MyCustomOApp { /* ... */ }
```

Remediation

Update the documentation so ownership is shown on the proper item type.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.